



Floosak Wallet API Documentation for Merchant Payment Integration

Payment Integration Guide

API Version	v1
Document Version	1.0
Last Updated	march 3, 2026
Environments	Sandbox, Production
Audience	External Providers, Merchant, and Integration Developers

Document History / Approval

Author	Status/Comment	Date	Version
Mohammed Halif	Initial Document	march 3, 2026	1.0
Ammar Alnouirah	Reviewer	march 3, 2026	1.0
Sadeq Al-radami	Approver	march 3, 2026	1.0

Floosak Merchant Integration API Endpoints

This table lists all available endpoints for authentication and payment

Service Area	Endpoint	Description	Method
Authentication	<code>/api/v1/auth/login</code>	Authenticate merchant and retrieve access token	POST
Authentication	<code>/api/v1/auth/forget-password</code>	merchant Password Reset Verification	POST
Authentication	<code>/api/v1/auth/change-password</code>	Change password confirmation	POST
Payment	<code>/api/v1/merchant/p2mcl</code>	Creates a pending purchase	POST
Payment	<code>/api/v1/merchant/p2mcl/confirm</code>	Confirms a pending transaction	POST
Payment	<code>/api/v1/merchant/p2mcl/refund</code>	refund a completed transaction back to the customer's wallet	POST
Check Status	<code>/api/v1/merchant/check-status</code>	Merchant Request Tracking	Post

Introduction

Floosak Wallet API for Merchant Payments enables merchants to securely process wallet-based transactions. It covers merchant authentication, payment initiation, OTP verification, transaction confirmation, and refunds.

All endpoints return standardized JSON responses with status and transaction details, ensuring accuracy, traceability, and a seamless payment experience.

This documentation provides a complete guide for integrating with the Floosak Wallet API. It is designed to be fully self-explanatory, offering step-by-step instructions, technical references, and examples to enable seamless merchant integration.

Contents

Document History / Approval	1
Floosak Merchant Integration API Endpoints	2
Introduction	2
1. API Overview	5
2. System Architecture Overview	5
3. Merchant Integration Overview	5
4. Base URL (Sandbox & Production)	6
5. Headers & Request Standards	6
6. Idempotency & request_id Strategy	6
7. Full Payment Flow Diagram.....	6
Flow diagram Example	6
8. Merchant Authentication Module	7
Sign-In guide	7
Sign-In– Endpoint.....	7
Request Headers	7
Sign-In– Request Body Parameters	7
Request Example	7
Sign-In – Response Example	7
Usage After Sign-In	8
Forget Password & Change Password	8
Forget Password	8
Forget password – Endpoint.....	8
Request Headers.....	8
Request Body Parameters (forget-password)	8
Forget password – Example Request	8
Forget password – Successful Response (200).....	8
Forget password – Business Rules	8
Change Password	9
Change Password – Endpoint	9
Request Headers.....	9
Request Body Parameters (forget-password)	9
Change Password – Example Request.....	9
Change password –Successful Response (200)	9
Change password – Business Rules	9
9. Send Endpoint - Deep Technical Documentation.....	10
Request Headers	10
HTTP Example.....	10
Send Payment – Request Parameters Table	10

Send Payment – JSON Example.....	10
Send Payment – Response Example.....	10
Send Payment – Response Fields Explanation	11
10. Confirm Endpoint - Deep Technical Documentation	11
Confirm – Endpoint	11
Confirm – JSON Example	11
Confirm – Request Parameters Table.....	11
Test Environment Behavior	11
Confirm – JSON Example	12
Confirm – Response Fields Explanation	12
Transaction Status Lifecycle	12
OTP Security Flow Explanation.....	12
11. Refund Transaction Endpoint	13
Refund – Endpoint.....	13
Refund – Request Headers	13
Refund – Request Body Parameters.....	13
Example Request (Refund)	13
Refund – Note	13
Refund – Successful Response (200)	13
Refund – Error Response.....	14
Refund Flow.....	14
Refund – Business Rules.....	14
12. Check Status	15
Check Status – Endpoint.....	15
Check Status –Request Body Parameters.....	15
Example Request (Check Status)	15
Check status – Successful Response (200)	15
Check status – Error Response	16
Check Status – Business Rules.....	16
13. Error Handling & Standard Error Format.....	16
14. Validation Rules & Business Constraints	16
15. Localization Notes (Arabic + English status fields)	17
16. Security Best Practices (FinTech Standards)	17
17. Step-by-Step Merchant Integration Guide	17
18. Glossary of Technical Terms	18

1. API Overview

This guide describes the integration of the Floosak Wallet with merchant systems, allowing customers to pay from their wallet to the merchant's Floosak wallet directly through the merchant's application using an OTP-secured payment flow.

- Two-step payment flow: Send -> Confirm
- Bearer token authentication on all endpoints
- Idempotent initiation using request_id
- Arabic and English transaction status support
- JSON request/response payloads

2. System Architecture Overview

In this architecture, the merchant's backend acts as the integration layer between the merchant application and the Floosak Wallet API.

All payment requests are initiated from the merchant system, allowing customers to pay directly from their Floosak wallet to the merchant's wallet without leaving the merchant's platform.

The Floosak API handles transaction creation, OTP delivery, OTP verification, and final payment settlement securely over HTTPS.

Component	Role
Merchant Backend	Initiates payment requests from the merchant system and submits OTP confirmation on behalf of the customer
Floosak Wallet API	Receives merchant requests, creates pending transactions, validates OTP, and manages the end-to-end payment flow
OTP Service	Sends the one-time password (OTP) to the customer's registered phone number for transaction authorization
Wallet Core/Ledger	Processes the transaction, updates wallet balances, and transfers funds from the customer wallet to the merchant wallet

3. Merchant Integration Overview

Merchants connect their platforms to the Floosak Wallet API to facilitate wallet-based payments from customers to the merchant's wallet without requiring customers to leave the merchant application.

Transactions are initiated programmatically and authorized via OTP, ensuring a secure and seamless payment experience.

4. Base URL (Sandbox & Production)

Environment	Base URL
Sandbox	https://staging.fintech-expert.net
Production	https://api.example.com

API versioning is path-based. Current endpoints are under /api/v1/.

5. Headers & Request Standards

Header	Required	Example
Authorization	Yes	Bearer {access_token}
Content-Type	Yes	application/json
Accept	Yes	application/json
x-channel	Yes	merchant

Requests and responses use JSON. Send UTF-8 encoded text for Arabic/English descriptions.

6. Idempotency & request_id Strategy

Use a unique request_id for each payment initiation. This prevents duplicate pending purchases when retrying the same Send request after network timeouts.

- Generate request_id using UUID/ULID.
- Persist request_id with purchase_id and reference_id for reconciliation

7. Full Payment Flow Diagram

Flow diagram Example

```
Merchant Backend -> POST /auth/sign-in -> Wallet API
Wallet API -> Response (Bearer Token)
Merchant Backend -> POST /purchases/send -> Wallet API
Wallet API -> OTP Service -> Customer Phone (OTP sent)
Wallet API -> Response (Pending + purchase_id)
Merchant UI <- Customer enters OTP
Merchant Backend -> POST /purchases/confirm (otp + purchase_id) ->
Wallet API
Wallet API -> Validate OTP + finalize payment -> Completed
Wallet API -> Response (Completed + reference_id)
```

8. Merchant Authentication Module

This section describes merchant authentication and credential management endpoints required before accessing protected payment APIs.

All authentication endpoints return JSON responses and must be consumed over HTTPS.

Sign-In guide

Authenticates the merchant and returns an access token (Bearer Token) used to authorize subsequent API requests.

Sign-In– Endpoint

POST /api/v1/auth/login

Request Headers

Content-Type: application/json
Accept: application/json
x-channel: merchant

Sign-In– Request Body Parameters

Name	Type	Required	Description
phone	string	Yes	Merchant phone number including country code (digits only).
password	string	Yes	Merchant password.

Request Example

```
{
  "phone": "9677XXXXXXX",
  "password": "XXXXXX"
}
```

Sign-In – Response Example

```
{
  "data": {
    "token": "ACCESS_TOKEN",
    "wallets": [
      {
        "id": 538704,
        "currency": {"id": 1, "name": { "en": "Yemeni Rial", "ar": "ريال يمني" }}
      ]
    },
    "balance": 500,
    "wallet_type_id": 1
  },
  "has_missing_data": "approved",
  "message": "تم تسجيل الدخول بنجاح",
  "is_success": true,
}
```

Usage After Sign-In

The returned token must be included in all secured requests:

```
Authorization: Bearer {token}
```

Forget Password & Change Password

These endpoints allow merchants to reset their password securely using OTP verification. The process is divided into two steps: Forget Password (request OTP) and Change Password (submit a new password with OTP).

Forget Password

Initiates password recovery by sending an OTP to the merchant's registered phone number.

Forget password – Endpoint

```
POST /api/v1/auth/forget-password
```

Request Headers

```
Content-Type: application/json
Accept: application/json
x-channel: merchant
```

Request Body Parameters (forget-password)

Name	Type	Required	Description
phone	string	Yes	Merchant phone number including country code (digits only).
identity_number	string	Yes	Official identity document number

Forget password – Example Request

```
{
  "phone": "9677XXXXXXX",
  "identity_number": "111xxxxxxxxx"
}
```

Forget password – Successful Response (200)

```
{
  "is_success": true,
  "message": "تم إرسال الرمز الخاص بالتحقق",
}
```

Forget password – Business Rules

- System validates merchant identity
- OTP is sent to the registered phone number
- OTP must be used in Change Password endpoint
- OTP has limited validity duration

Change Password

Changes the merchant password after OTP verification.

Change Password – Endpoint

POST /api/v1/auth/change-password

Request Headers

Content-Type: application/json
Accept: application/json
x-channel: merchant

Request Body Parameters (forget-password)

Name	Type	Required	Description
phone	string	Yes	Merchant phone number including country code (digits only).
otp	integer	Yes	One-Time Password received via SMS
new_password	string	Yes	New password to replace the old password

Change Password – Example Request

```
{
  "phone": 9677XXXXXXX,
  "otp": 000000,
  "new_password": 112233
}
```

Change password –Successful Response (200)

```
{
  "is_success": true,
  "message": "Password changed successfully"
}
```

Change password – Business Rules

- OTP must match the last generated OTP
- Password must meet system-defined security rules (e.g at least 6 character).
- After a successful change, the OTP becomes invalid and cannot be reused.

9. Send Endpoint - Deep Technical Documentation

Creates a pending purchase and triggers OTP to the customer phone.

Request Headers

Authorization: Bearer <access_token>
Content-Type: application/json
Accept: application/json
x-channel: merchant

HTTP Example

```
POST /api/v1/merchant/p2mcl
```

Send Payment – Request Parameters Table

Name	Type	Required	Description
source_wallet_id	Integer	Yes	Merchant wallet ID
request_id	string	Yes	Unique idempotency key
target_phone	number/string	Yes	Customer phone number with country code
amount	Number	Yes	Payment amount
purpose	string	Yes	Payment description (Arabic/English supported)

Send Payment – JSON Example

```
{
  "source_wallet_id": 25312504,
  "request_id": "//Unique UUID generated by the merchant system for each request",
  "target_phone": "9677XXXXXXX",
  "amount": 100,
  "purpose": "Payment description"
}
```

Important: The returned data.id is the purchase_id used later in the Confirm endpoint.

Send Payment – Response Example

```
{ "data": { "name": "Customer Name",
  "balance": 300,
  "phone": "967XXXXXXX",
  "status": { "ar": "معلقه", "en": "Pending" },
  "net": 100,
  "fee": 1,
  "gross": 101,
  "reference_id": "4633395571756870",
  "id": 1699240, //This id will be used in the next Confirm request as purchase_id
  "created_at": "2026-02-22T10:41:11"},
  "is_success": true }
```

Send Payment – Response Fields Explanation

Field	Meaning
net	Base transaction amount
fee	Applied service fee
gross	Total amount including fee
id	Record identifier (Send response id is purchase_id for confirmation)
status.ar / status.en	Localized transaction status labels
balance	Returned wallet balance value in response context

10. Confirm Endpoint - Deep Technical Documentation

The **Confirm Endpoint** is used to finalize a pending purchase transaction by validating the **One-Time Password (OTP)** provided by the customer.

After successful validation, the transaction is executed and the wallet balance is updated.

Confirm – Endpoint

```
POST /api/v1/merchant/p2mcl/confirm
```

Confirm – JSON Example

```
{
  "otp": 123456,
  "purchase_id": 1699240 //Use the ID returned from the Send request
}
```

Confirm – Request Parameters Table

Field	Type	Required	Description
otp	number/string	Yes	Customer OTP code
purchase_id	integer	Yes	Pending purchase ID returned by Send

Test Environment Behavior

In the sandbox/test environment, the OTP value is always **123456**.

No OTP message will be sent to the customer. This is implemented to simplify testing during integration.

Confirm – JSON Example

```
{  "data": {
    "name": "Customer Name",
    "balance": 400,
    "phone": "967XXXXXXXXX",
    "status": {"ar": "مكتملة", "en": "Completed"},
    "net": 100,
    "fee": 1,
    "gross": 101,
    "reference_id": "4633391771756870",
    "id": 620453,
    "created_at": "2026-02-22T10:56:35"
  },
  "is_success": true}
```

Confirm – Response Fields Explanation

Field	Meaning
net	Base transaction amount
fee	Applied service fee
gross	Total amount including fee
reference_id	Wallet platform transaction reference for support/reconciliation
status.ar / status.en	Localized transaction status labels
balance	Returned wallet balance value in response context

Transaction Status Lifecycle

Status	Arabic	English	Meaning
Pending	معلقة	Pending	OTP sent; waiting for confirmation
Completed	مكتملة	Completed	OTP verified; transaction finalized

Do not mark merchant orders as paid while the transaction is still Pending.

OTP Security Flow Explanation

OTP adds a customer confirmation step before finalizing payment. The OTP should be treated as sensitive and never stored or logged in plaintext.

- Collect OTP only when Send returns success and Pending status
- In the test Environment The value **123456** must always be used as the OTP when calling this endpoint.
- Submit OTP over HTTPS from merchant backend

11. Refund Transaction Endpoint

This endpoint allows a merchant to refund a completed transaction back to the customer's wallet. The refund request must reference the original transaction ID obtained from the Confirm endpoint.

Refund – Endpoint

```
POST /api/v1/merchant/p2mcl/refund
```

Refund – Request Headers

Authorization: Bearer <access_token>
Content-Type: application/json
Accept: application/json
x-channel: merchant

Refund – Request Body Parameters

Name	Type	Required	Description
transaction_id	integer	Yes	The ID of the original completed transaction (from Confirm endpoint).
request_id	String	Yes	Unique idempotency key
amount	number	Yes	Amount to refund. Must not exceed the original transaction net amount.

Example Request (Refund)

```
{  "transaction_id": 620492,
  "request_id": //Unique UUID generated by the merchant system for each request,
  "amount": 100}
```

Refund – Note

Note: transaction_id must match the id returned from the Confirm endpoint for the original transaction.

Refund – Successful Response (200)

```
{  "data": {
    "name": "صيفان علي مهدي سعيد",
    "balance": 500,
    "phone": "967772413444",
    "status": {"ar": "مكتملة", "en": "Completed"},
    "net": 100,
    "fee": 0,
    "rate": 0,
    "gross": 100,
    "reference_id": "Refund",
    "id": 620493,
    "created_at": "2026-03-01T10:49:03.000000Z"
  },
  "message": "تمت بنجاح",
  "is_success": true }
```

Refund – Error Response

```
{
  "message": "This transaction does not exist",
  "errors": {"transaction": ["This transaction does not exist"]},
  "statusCode": 2
}
```

Refund Flow

- Merchant retrieves id from the Confirm endpoint of the original transaction.
- Merchant sends a refund request using transaction_id and amount.
- System validates that the transaction_id exists and the amount does not exceed the original transaction.
- If validation passes, the refund is applied to the customer's wallet, and a new refund transaction record is created.
- Response includes updated balance and refund transaction details.

Refund – Business Rules

- Only **Completed** transactions can be refunded.
- The refund amount must be less than or equal to the original transaction net amount.
- transaction_id must be valid and belong to the authenticated merchant.
- Each refund request must include a unique request_id.
- Refunds do not generate a new OTP; they are immediate and final.

12. Check Status

This endpoint allows the merchant to verify the status of a previously submitted transaction using the request_id. It is used to confirm whether the operation has been successfully completed, is still pending, or has failed.

Check Status – Endpoint

```
POST /api/v1/merchant/check-status
```

```
Authorization: Bearer <access_token>
Content-Type: application/json
Accept: application/json
x-channel: merchant
```

Check Status –Request Body Parameters

Name	Type	Required	Description
request_id	String	Yes	Unique idempotent reference used when the transaction was created.

Example Request (Check Status)

```
{
  "request_id": " 538681771877797"
}
```

Check status – Successful Response (200)

```
{
  "data": "transaction retrieved successfully",
  "message": {
    "name": ".....",
    "balance": 76449.13,
    "phone": "9677XXXXXXX",
    "currency": { "en": "YER", "ar": "ر.ي." },
    "status": { "ar": "مكتملة", "en": "Completed" },
    "net": 1200,
    "fee": 0,
    "rate": 0,
    "gross": 1200,
    "data": {
      "cached_by_name": " name response",
      "cached_by_phone": "9677XXXXXXX"},
    "purpose": "for api test",
    "id": 628293,
    "wallet_id": 2262,
    "target_nickname": " name response",
    "target_phone": "9677XXXXXXX",
  },
  "is_success": true,
  "statusCode": 1}
```

Check status – Error Response

```
{
  "message": "This transaction does not exist",
  "errors": {"transaction": ["This transaction does not exist"]},
  "statusCode": 2
}
```

Check Status – Business Rules

- The request_id must exist and belong to the authenticated merchant.
- The endpoint is read-only and does not modify any transaction data.

13. Error Handling & Standard Error Format

Errors are returned as JSON with an HTTP status code and a stable error code for programmatic handling.

HTTP	Example statusCode	Meaning
401	AUTH_INVALID_TOKEN	Missing or invalid bearer token
422	VALIDATION_ERROR	Payload validation failed
409	DUPLICATE_REQUEST_ID	Duplicate Send request_id
404	PURCHASE_NOT_FOUND	Unknown purchase_id
429	RATE_LIMIT_EXCEEDED	Too many requests
500	Server error	Unexpected server error
2	statusCode	Return response with error

14. Validation Rules & Business Constraints

- source_wallet_id must belong to the authenticated merchant
- target_phone must be a valid phone number with country code
- amount must be positive and within allowed limits
- purchase_id must reference a pending transaction during Confirm
- purpose should be concise and user-safe text

15. Localization Notes (Arabic + English status fields)

The API returns status labels in Arabic and English. Use `status.en` for backend logic and `status.ar` for Arabic UI display where needed.

JSON Example

```
{
  "status": { "ar": "معلقه" , "en": "Pending" }
}
```

16. Security Best Practices (FinTech Standards)

- Use HTTPS/TLS for all environments
- Protect bearer tokens and rotate on suspected compromise
- Never log OTP values
- Store request/response timestamps and IDs for auditability
- The `target_phone` used in the payment request must match the customer's phone number already registered in the merchant system.
- The merchant must not allow manual entry of a different phone number during checkout if the customer is already registered.

17. Step-by-Step Merchant Integration Guide

1. Configure API base URL (Sandbox or Production).
2. Call POST `/api/v1/auth/sign-in` to obtain a bearer token.
3. Store the returned Bearer token and include it in the Authorization header for all requests.
4. Generate a unique `request_id` for the payment initiation.
5. Call POST `/api/v1/merchant/p2mcl` and store `data.id` as `purchase_id`.
6. Collect OTP from the customer.
7. Call POST `/api/v1/merchant/p2mcl/confirm` with `otp` and `purchase_id`.
8. Mark order as paid only when `status.en` is Completed.
9. If a refund is required, generate a new unique `request_id`.
10. Call POST `/api/v1/merchant/p2mcl/refund` using the confirmed transaction `id` as `transaction_id`.
11. Consider the refund successful only when `is_success` is true and `status.en` is Completed.
12. If the merchant forgets the password, call POST `/api/v1/auth/forget-password`.
13. Call POST `/api/v1/auth/change-password` with `otp` and the new password to complete password reset.

18. Glossary of Technical Terms

Term	Definition
purchase_id	Unique identifier of a pending purchase returned from the Send endpoint; required for Confirm.
transaction_id	Identifier of a completed transaction used when initiating a Refund request.
reference_id	Platform-generated transaction reference used for reconciliation, reporting, and support inquiries.
request_id	Merchant-generated unique idempotency key used to prevent duplicate Send or Refund requests.
net	Base transaction amount before fees.
fee	Transaction fee charged by the platform (if applicable).
gross	Total transaction amount including fees (net + fee).
balance	Merchant wallet balance after the transaction or refund is processed.
OTP	One-Time Password sent to the customer's registered phone number for transaction confirmation.
Pending	Transaction status indicating that OTP confirmation is still required.
Completed	Transaction status indicating the operation has been successfully finalized.

QUALITYCONNECT
كواليتي كونكت